

Reporte tecnico: Observabilidad distribuida con OpenTelemetry en AWS

1. Resumen ejecutivo

Este laboratorio implementa una arquitectura de dos microservicios instrumentados con OpenTelemetry y desplegados en AWS ECS Fargate. El objetivo fue demostrar los tres pilares de observabilidad: trazas, metricas y logs correlacionados. La solucion usa `service-a` como servicio de entrada, `service-b` como servicio dependiente por HTTP y DynamoDB como base de datos administrada.

La implementacion se hizo con Quarkus, OpenTelemetry SDK, OpenTelemetry Collector, AWS X-Ray, Jaeger, CloudWatch Logs y Grafana. La infraestructura se definio con Terraform para permitir despliegue reproducible, bajo costo y destruccion controlada al finalizar el taller.

La evidencia recolectada demuestra propagacion de contexto entre servicios, trazas distribuidas completas, logs JSON con `trace\_id` y `span\_id`, metricas ECS visibles en Grafana y un benchmark comparativo entre ejecucion sin OTel y con OTel.

2. Arquitectura objetivo

La arquitectura funcional implementada sigue el flujo:

```
```\ntext
cliente/k6 -> service-a -> service-b -> DynamoDB
\\`
```

`service-a` expone el endpoint `GET /orders/{id}`. Al recibir una solicitud, crea o continua el contexto de traza, ejecuta logica de negocio en el span `process-order` y llama a `service-b` usando HTTP. `service-b` expone `GET /products/{id}`, ejecuta el span `lookup-product` y consulta DynamoDB para devolver el producto asociado.

En AWS, ambos servicios corren como ECS Fargate Services dentro del cluster `otel-lab-cluster`. Cada task de aplicacion incluye un collector como sidecar:

```
```\ntext
Task service-a: service-a + otel-collector-a
Task service-b: service-b + otel-collector-b
\\`
```

Adicionalmente se desplegaron servicios ECS para Grafana y Jaeger. Jaeger recibe trazas OTLP desde los collectors mediante Cloud Map en `jaeger.otel-lab.local:4317`. Grafana consume CloudWatch como datasource para logs y metricas, y Jaeger como datasource de trazas.

3. Instrumentacion OpenTelemetry

La instrumentacion se implemento usando Quarkus OpenTelemetry. Se habilito instrumentacion automatica para HTTP inbound, HTTP outbound y DynamoDB mediante el SDK/extension de Quarkus. La propagacion entre `service-a` y `service-b` usa W3C Trace Context, permitiendo que ambos servicios compartan el mismo `trace\_id`.

Tambien se agregaron spans personalizados para la logica critica:

- `process-order` en `service-a`.
- `lookup-product` en `service-b`.

Los spans incluyen atributos funcionales como:

- `order.id`

- `product.id`
- `business.operation`
- `db.system=dynamodb`
- `db.name=otel-lab-products`

Esto permite filtrar trazas en Jaeger por identificadores de negocio como `order.id=order-1` o `product.id=product-3`.

4. Logs estructurados y correlacion

Los servicios emiten logs JSON hacia stdout. En AWS, esos logs son recolectados por CloudWatch Logs en:

- `/otel-lab/service-a`
- `/otel-lab/service-b`

Cada linea relevante contiene `trace\_id` y `span\_id`, lo que permite correlacionar logs con trazas. La evidencia documentada en `docs/evidence/cloudwatch-log-correlation.md` usa el trace:

```
```text
ecdcaea8c31b1d951c53d0f59a6fa461
```
```

Ese mismo `trace\_id` aparece en logs de `service-a` y `service-b`, validando la propagacion de contexto sobre la llamada HTTP interna.

5. OpenTelemetry Collector

El collector AWS se configuro con:

- Receiver OTLP gRPC y HTTP.
- Processor `memory\_limiter`.
- Processor `resource`.
- Processor `batch`.
- Exporter `awsxray`.
- Exporter `otlp/jaeger`.
- Exporter `awscloudwatchlogs`.
- Exporter `debug`.

Las trazas se exportan simultaneamente a AWS X-Ray y Jaeger:

```
```text
traces -> awsxray, otlp/jaeger, debug
```
```

Los logs OTLP se exportan a CloudWatch Logs:

```
```text
logs -> awscloudwatchlogs, debug
```
```

El collector tambien expone metricas internas por Prometheus en el ambiente local y genera senales de salud observables mediante logs del collector en CloudWatch.

6. Backends de visualizacion

AWS X-Ray

AWS X-Ray se uso como backend administrado de trazas en AWS. La evidencia tomada muestra el flujo:

```
```text
service-a -> service-b -> DynamoDB
```
```

Esto valida que las trazas llegan desde los collectors y que la propagacion de contexto funciona entre servicios.

Jaeger

Jaeger se desplego en ECS Fargate para cubrir el requerimiento de visualizacion de trazas completas. Los collectors exportan a Jaeger por OTLP gRPC usando Cloud Map. La UI de Jaeger permite buscar por servicio y por tags como `order.id` y `product.id`.

Grafana

Grafana se desplego en ECS Fargate y se provisiono con dos datasources:

- CloudWatch para metricas/logs.
- Jaeger para trazas.

El dashboard AWS contiene seis paneles:

1. CPU por ECS Service.
2. Memoria por ECS Service.
3. SLI de requests de `service-a`.
4. SLI de lookups de `service-b`.
5. Errores de aplicacion.
6. Senal de salud/logs del collector.

Las metricas de CPU y memoria se presentan como tabla de evidencia basada en CloudWatch `AWS/ECS` con dimension `ServiceName`.

7. Infraestructura como codigo

Terraform define los recursos principales:

- VPC, subnet publica, route table e Internet Gateway.
- Security groups para servicios, Grafana y Jaeger.
- ECS cluster y ECS services.
- Task definitions Fargate.
- Cloud Map para resolucion privada.
- DynamoDB `otel-lab-products`.
- ECR repositories.
- IAM roles y policies.
- CloudWatch log groups.
- AWS Budget.

Se priorizo bajo costo evitando NAT Gateway, ALB, EKS, RDS, Amazon Managed Grafana y Amazon Managed Prometheus. Para un taller academico, exponer `service-a`, Grafana y Jaeger mediante IP publica de Fargate fue una decision pragmatica. En produccion se reemplazaria por ALB/API Gateway, dominios DNS y controles de acceso.

8. Analisis de overhead

El benchmark local se ejecuto con k6 usando 10 VUs, 30 segundos de warmup y 2 minutos de carga estable. El flujo medido fue `service-a -> service-b -> DynamoDB Local`.

| | Without OTel | With OTel | Observed overhead |
|--|--------------|-----------|-------------------|
|--|--------------|-----------|-------------------|

| | Without OTEL | With OTEL | Difference |
|-----------------|--------------|-------------|------------|
| Average latency | 32.73 ms | 70.30 ms | +114.78% |
| p50 latency | 29.33 ms | 41.24 ms | +40.61% |
| p95 latency | 65.30 ms | 166.61 ms | +155.15% |
| p99 latency | 96.92 ms | 508.31 ms | +424.47% |
| Throughput | 8.133 req/s | 7.862 req/s | -3.33% |
| Error rate | 0.06% | 0.00% | -0.06 pp |

Los resultados muestran overhead de latencia en el escenario instrumentado. La latencia p99 aumento de 96.92 ms a 508.31 ms, equivalente a +424.47%. Aun asi, el p99 se mantuvo por debajo del umbral definido de 1500 ms y la tasa de errores permanecio dentro del objetivo de `http\_req\_failed < 1%`.

La muestra puntual de CPU/memoria mostro mayor uso de CPU de aplicacion con OTEL y memoria adicional de collectors:

| Component | Without OTEL | With OTEL | Difference |
|------------------------|----------------|------------------|------------|
| App CPU total | 0.95% | 2.92% | +207.37% |
| App memory total | 420.9 MiB | 408.8 MiB | -12.1 MiB |
| Collector memory total | 82.47 MiB idle | 96.39 MiB active | +13.92 MiB |

9. Decisiones de disenio

Se eligio Quarkus por su soporte nativo para OpenTelemetry, REST Client, Micrometer y AWS SDK. Se eligio DynamoDB en lugar de RDS para reducir costo, simplificar IAM y evitar redes privadas complejas. Se eligio ECS Fargate porque elimina administracion de nodos y permite tareas efimeras de bajo costo.

El collector como sidecar mantiene baja latencia local para OTLP y simplifica permisos por task role. Jaeger y Grafana como servicios ECS adicionales permiten cumplir el entregable dentro del mismo ambiente AWS sin depender solo de herramientas locales.

10. Conclusiones

El laboratorio cumple la arquitectura objetivo de dos microservicios con dependencia HTTP y acceso a base de datos. La instrumentacion OTEL genera trazas, metricas y logs correlacionables. AWS X-Ray y Jaeger validan trazas distribuidas completas. CloudWatch y Grafana permiten visualizar logs, SLIs y metricas ECS.

El analisis de overhead indica que, bajo la carga probada, la instrumentacion incremento la latencia de cola, especialmente p99. El costo principal se observa en la latencia adicional, CPU asociada a la instrumentacion y memoria de los collectors. Para una medicion mas rigurosa se recomienda ejecutar pruebas mas largas, alternar orden de ejecucion, reiniciar contenedores entre corridas y usar multiples repeticiones estadisticas.

11. Evidencias

Las capturas se encuentran en `docs/screenshots/` e incluyen:

- X-Ray service graph.
- X-Ray trace completa.
- Jaeger trace completa.
- Grafana dashboard de seis paneles.
- Logs correlacionados por `trace\_id`.
- ECS services en estado running.

La correlacion de logs se documento en `docs/evidence/cloudwatch-log-correlation.md`.